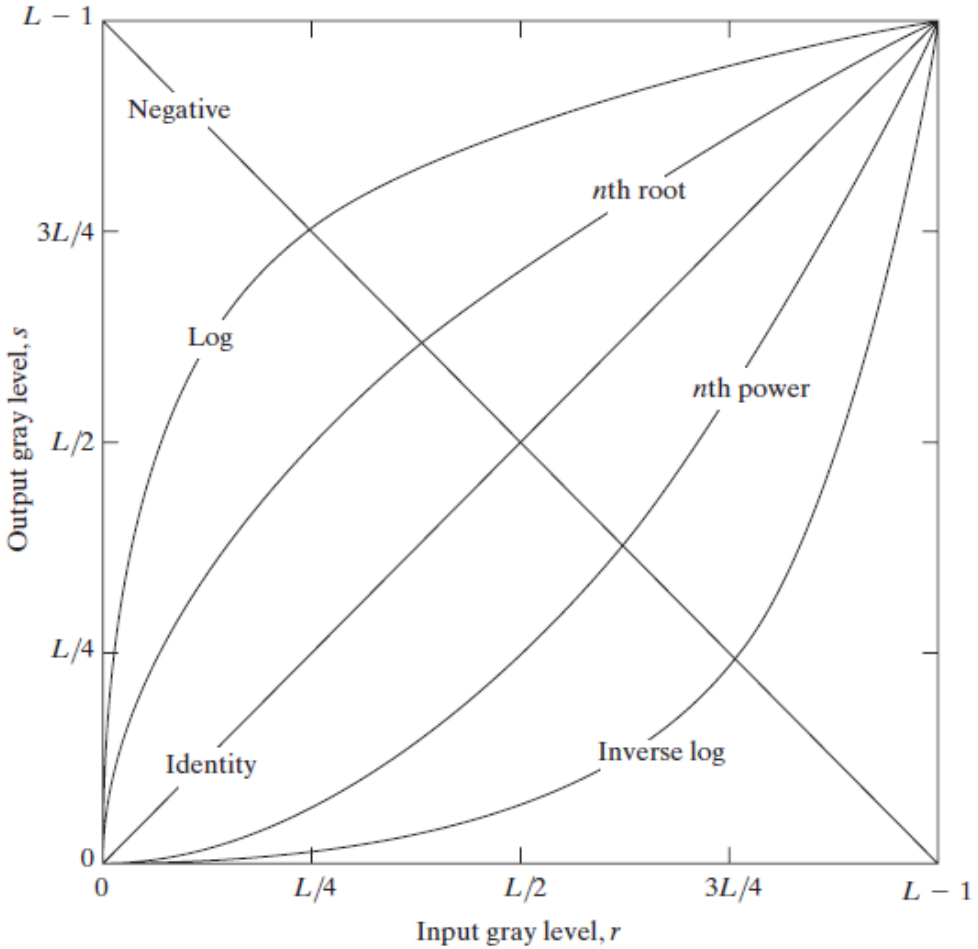
INTRODUCTION TO IMAGE PROCESSING

LECTURE 3
Image Enhancement in Spatial Domain

Gray Level (Intensity) Transformations

Basic gray-level transformation functions used for image enhancement.



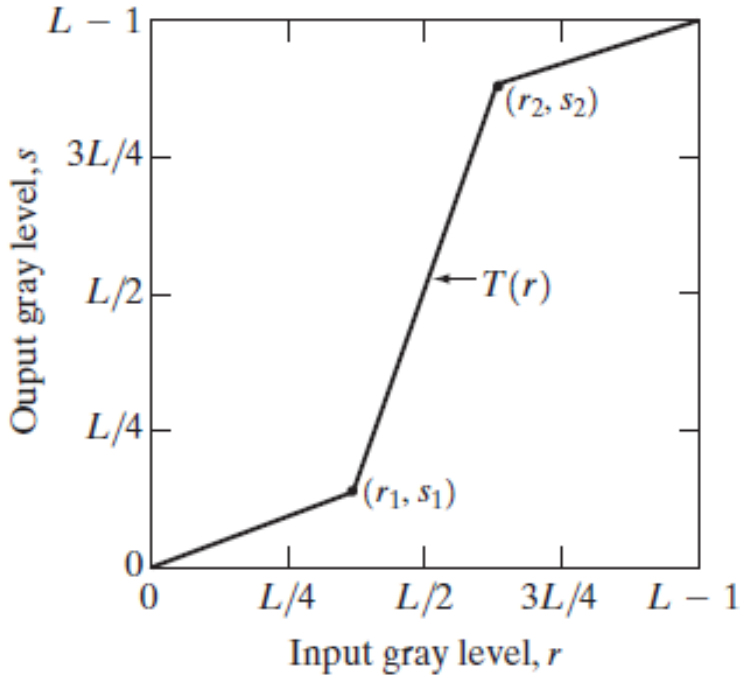
$$s = T(r)$$

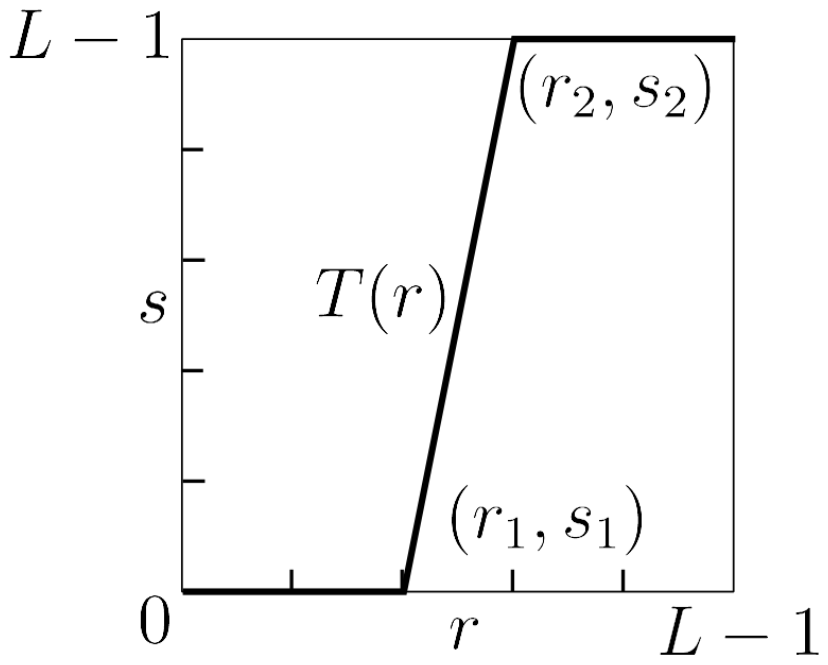
- Linear transformations (negative and identity transformations)
- Logarithmic transformations (log and inverse-log transformations)
- Power-law transformtaions (*n*th power and *n*th root transformations) (gamma transformations)

Gray Level (Intensity) Transformations

Piecewise-Linear Transformation Functions

- One of the simplest piecewise linear functions is a **contrast-stretching transformation**.
- Low-contrast images can result from poor illumination, lack of dynamic range in the imaging sensor, or even wrong setting of a lens aperture during image acquisition.
- With contrast stretching, we can increase the dynamic range of the gray levels in the image.

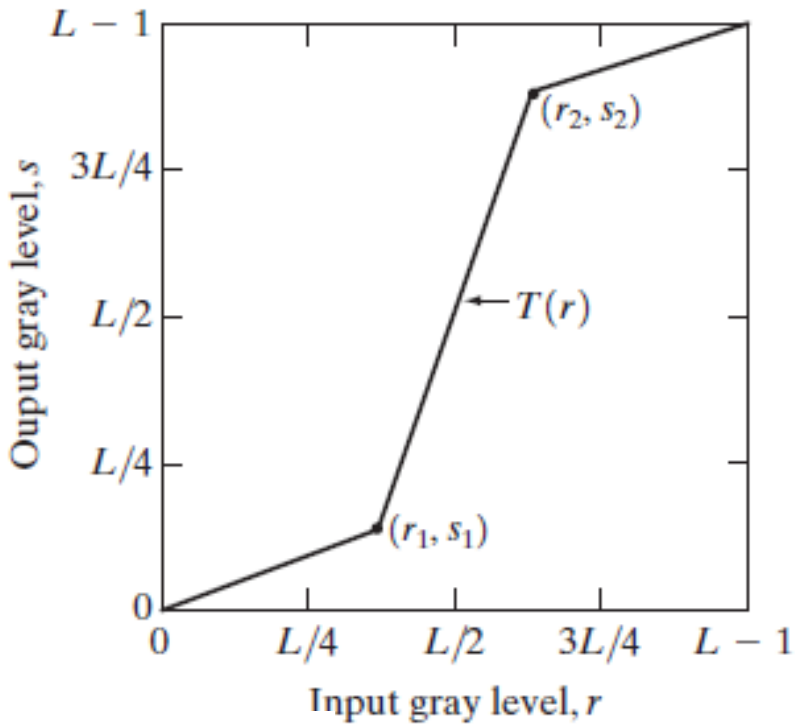




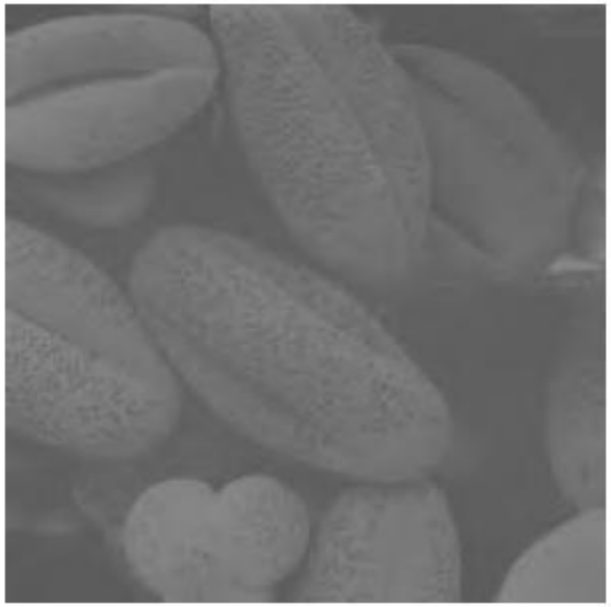
$$s = T(r)$$

Gray Level (Intensity) Transformations

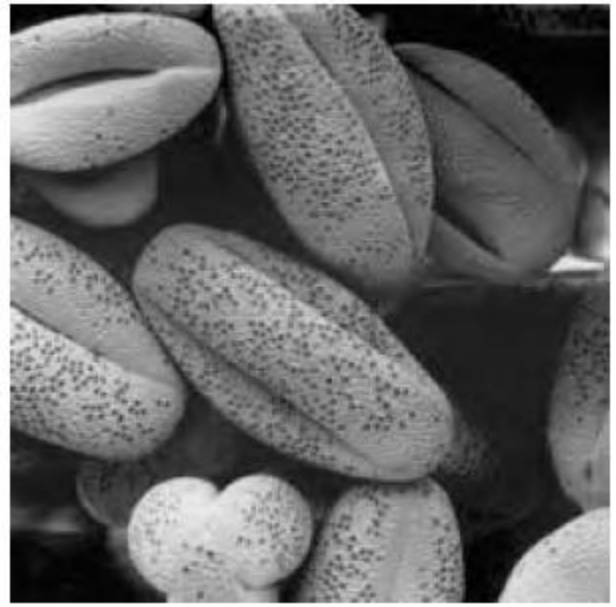
Contrast-stretching with piecewise linear functions



$$s = T(r)$$



A low-contrast image



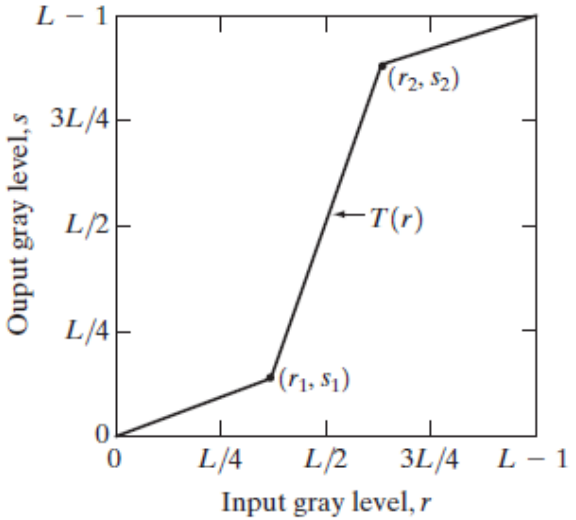
Result of contrast stretching.

$$(r_1, s_1) = (r_{min}, 0)$$
$$(r_2, s_2) = (r_{max}, L-1)$$

r_{min} and r_{max} denote the minimum and maximum gray levels in the image, respectively. Thus, the transformation function stretched the levels linearly from their original range to the full range $[0, L-1]$.

Gray Level (Intensity) Transformations

Contrast Stretching



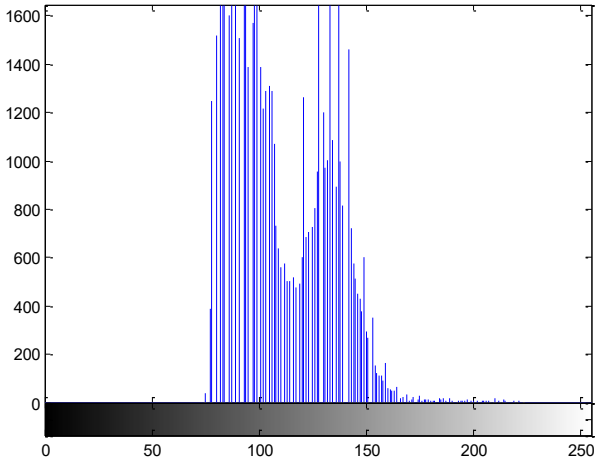
$$s = T(r)$$

$$(r_1, s_1) = (r_{min}, 0)$$
$$(r_2, s_2) = (r_{max}, L-1)$$

r_{min} and r_{max} denote the minimum and maximum gray levels in the image, respectively. Thus, the transformation function stretched the levels linearly from their original range to the full range $[0, L-1]$.

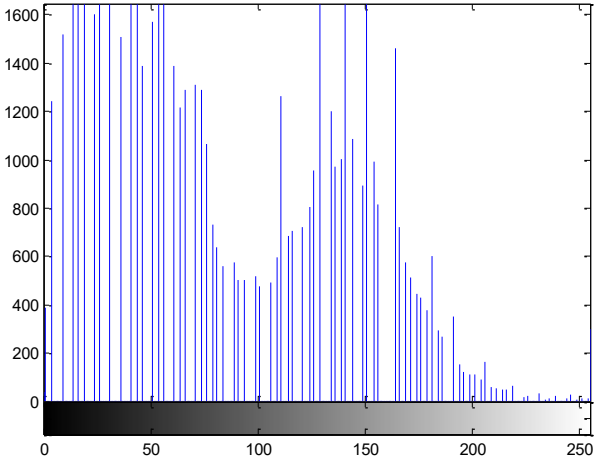


A low-contrast image





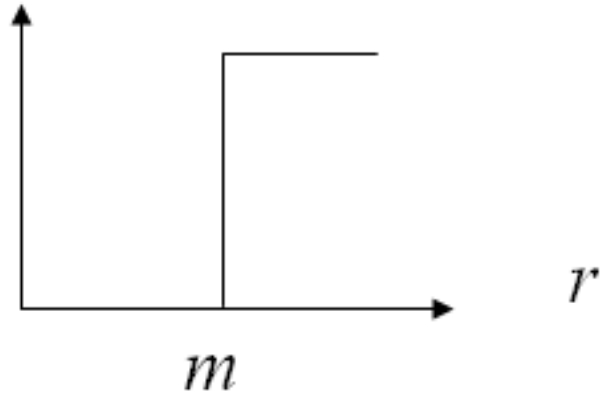
Result of contrast stretching.



Gray Level (Intensity) Transformations

Thresholding (Binarization)

$s = T(r)$

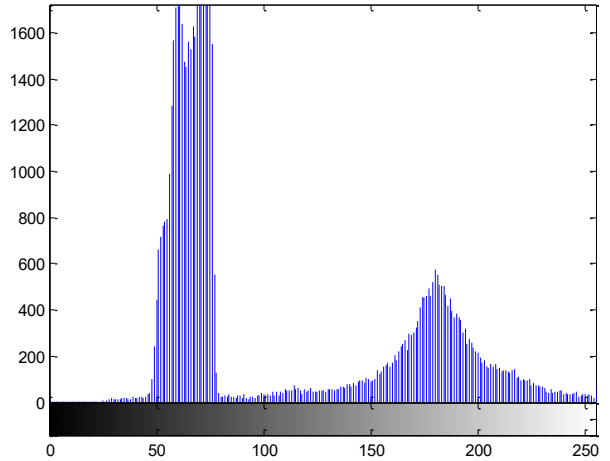


$$s = T(r) = \begin{cases} 1 & r \geq m \\ 0 & r < m \end{cases}$$

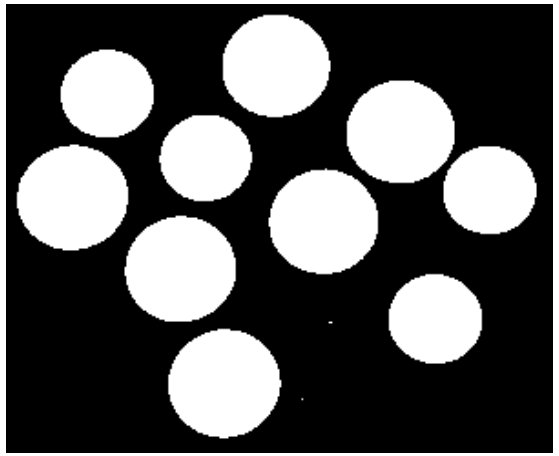
$$g(x, y) = T[f(x, y)]$$



Input image $f(x, y)$



Histogram of the input image

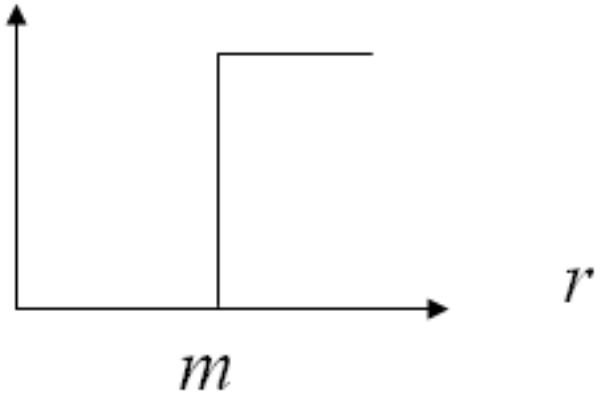


Output image $g(x, y)$

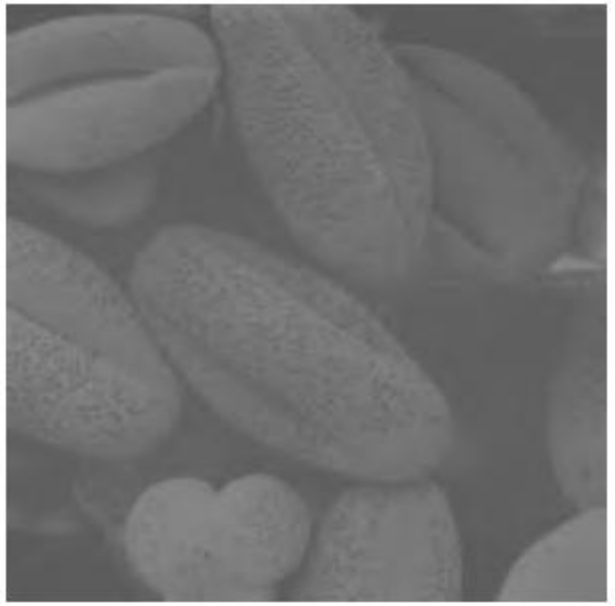
Gray Level (Intensity) Transformations

Thresholding (Binarization)

$s = T(r)$



$$s = T(r) = \begin{cases} 1 & r \geq m \\ 0 & r < m \end{cases}$$



Input image $f(x, y)$



Output image $g(x, y)$

$$g(x, y) = T[f(x, y)]$$

Image Histogram

Histogram $h(r_k) = n_k$

r_k is the kth intensity value

n_k is the number of pixels in the image with intensity r_k

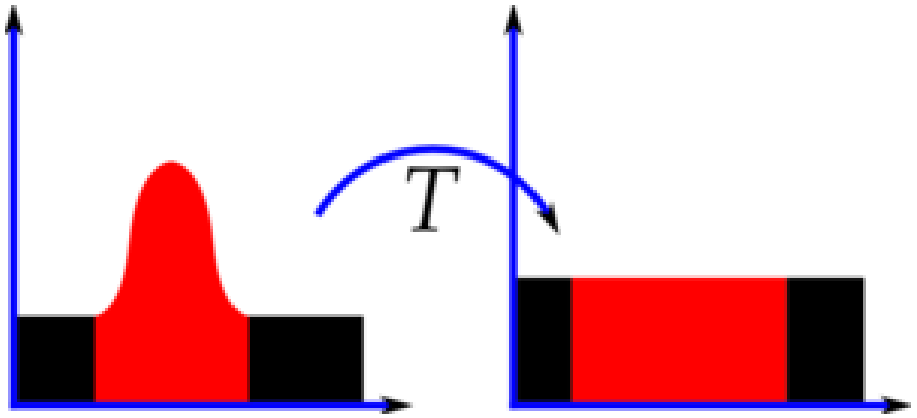
Normalized histogram $p(r_k) = \frac{n_k}{MN}$

n_k: the number of pixels in the image of size $M \times N$ with intensity r_k

Histogram Equalization

We want to find a transformation function that will stretch the histogram of the image to the full range of available gray levels [0, L-1], and that will result in a more or less uniform distribution of the gray levels.
In this way, we will obtain an image which will have an appearance of high contrast and will exhibit a large variety of gray tones.

$$s = T(r)$$





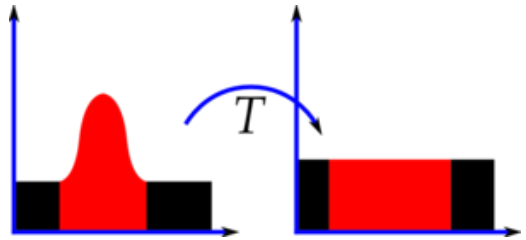
Original image *Bay*



... after histogram equalization

Histogram Equalization

$$s = T(r)$$



The probability of occurrence of gray level r_k in an image is approximated by

$$p_r(r_k) = \frac{n_k}{n} \qquad k = 0, 1, 2, \ldots, L-1$$

where n is the total number of pixels in the image, n_k is the number of pixels that have gray level r_k , and L is the total number of possible gray levels in the image.

The transformation function for histogram equalization is given by

$$s_k = T(r_k) = \sum_{j=0}^{k} p_r(r_j)$$

$$= \sum_{j=0}^{k} \frac{n_j}{n} \qquad k = 0, 1, 2, \ldots, L-1.$$

We map each pixel with level r_k in the input image into a corresponding pixel with level s_k in the output image.

Histogram Equalization

Output Range	Histogram Equalization Transformation
Normalized Range $[0, 1]$	$s_k = \sum_{j=0}^{k} p_r(r_j)$
Gray-Level Range $[0, L-1]$	$s_k = (L-1) \sum_{j=0}^{k} p_r(r_j)$

Histogram Equalization

Example

Suppose that a 3-bit image ($L = 8$) of size $M \times N = 64 \times 64$ pixels ($n = 4096$) has the intensity distribution (histogram) shown in following table.
We get the histogram equalization transformation function and give the $p_r(r_k)$ for each r_k.

r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

Histogram Equalization

Example - continued

$$s_k = T(r_k) = \sum_{j=0}^{k} p_r(r_j)$$

$$= \sum_{j=0}^{k} \frac{n_j}{n} \qquad k = 0, 1, 2, \ldots, L-1.$$

$$s_0 = T(r_0) = 7\sum_{j=0}^{0} p_r(r_j) = 7 \times 0.19 = 1.33 \rightarrow 1$$

$$s_1 = T(r_1) = 7\sum_{j=0}^{1} p_r(r_j) = 7 \times (0.19 + 0.25) = 3.08 \rightarrow 3$$

$$s_2 = 4.55 \rightarrow 5 \qquad s_3 = 5.67 \rightarrow 6$$

$$s_4 = 6.23 \rightarrow 6 \qquad s_5 = 6.65 \rightarrow 7$$

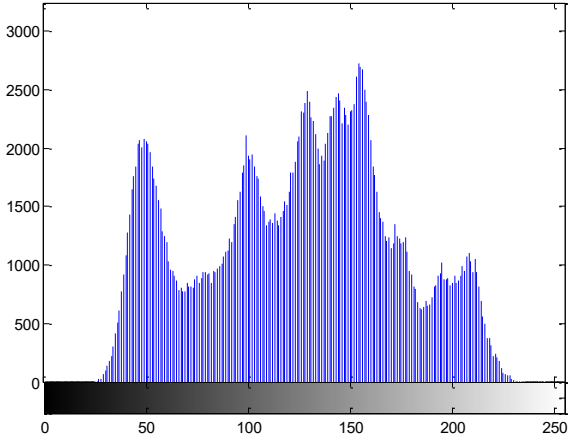
$$s_6 = 6.86 \rightarrow 7 \qquad s_7 = 7.00 \rightarrow 7$$

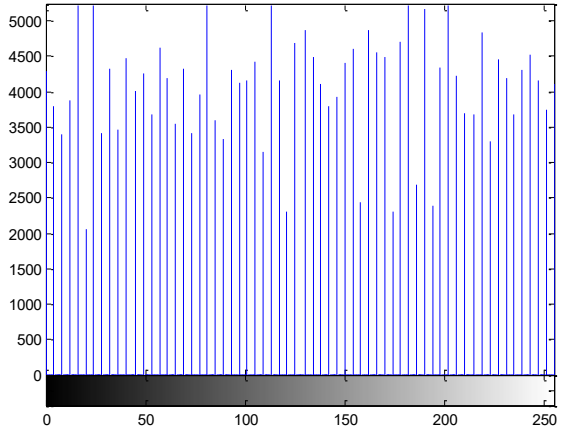
r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

Histogram Equalization









Histogram Equalization

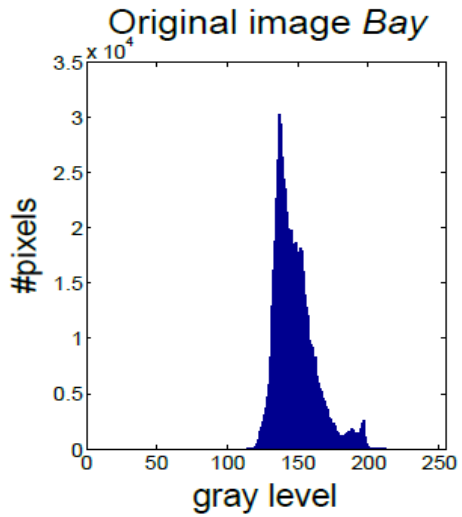


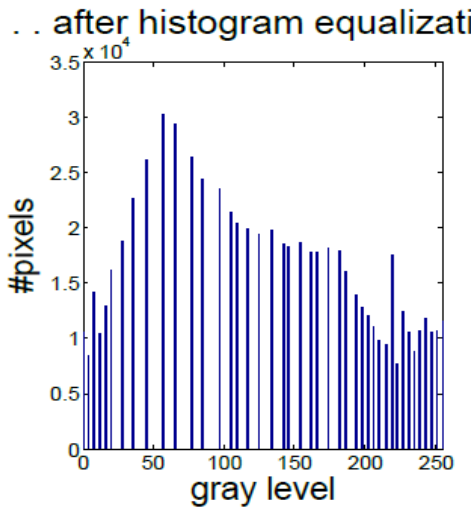
Original image *Bay*



... after histogram equalization



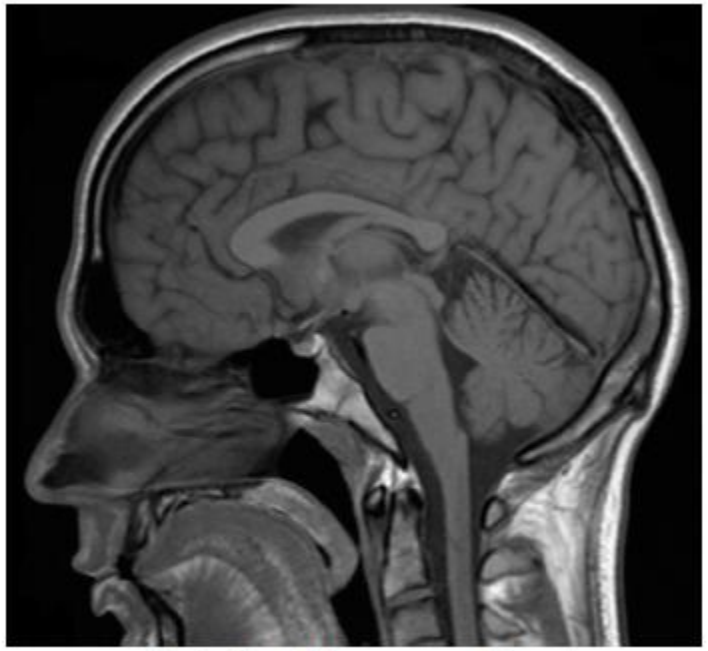




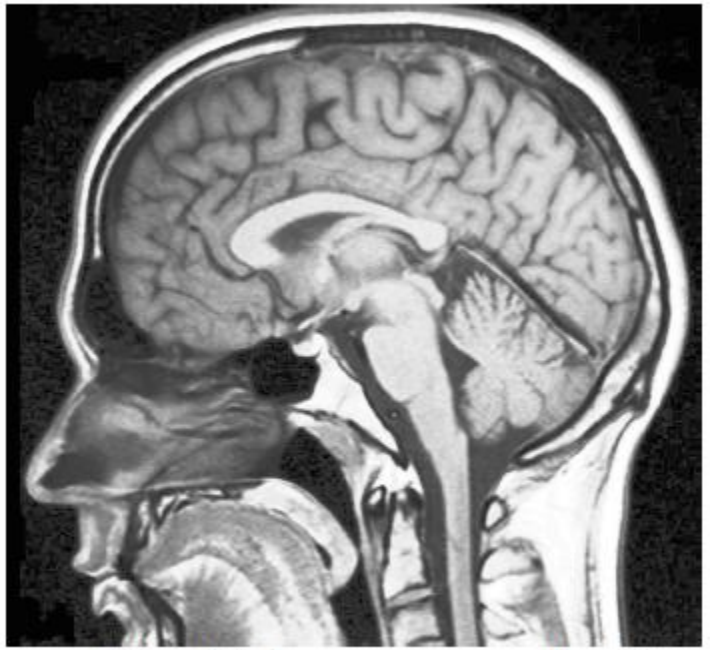


Histogram equalization increases the contrast.
The histogram of the resulting image spans the full spectrum of the gray scale.

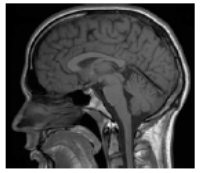
Histogram Equalization



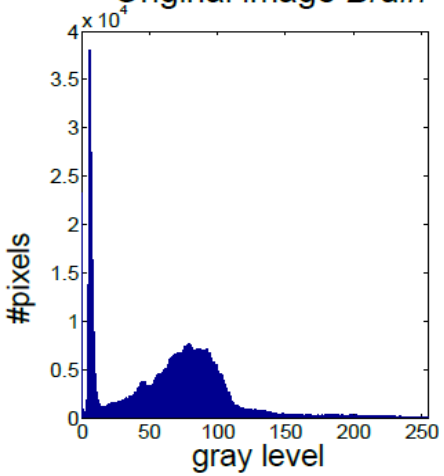
Original image *Brain*



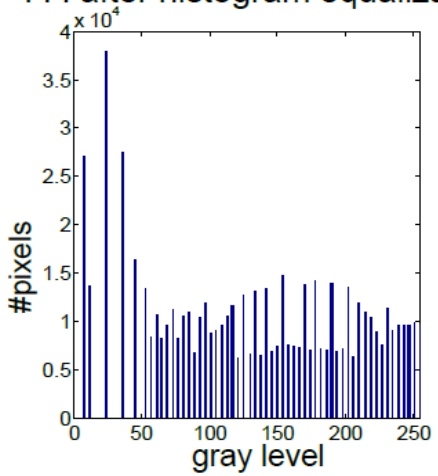
... after histogram equalization



Original image *Brain*



. . . after histogram equalization





Histogram Equalization

Example

$$I = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 2 & 2 & 2 \\ 2 & 3 & 3 & 3 \\ 3 & 3 & 4 & 7 \end{bmatrix}$$

Apply histogram equalization to the given image and determine the resulting image matrix.

1-Calculate the histogram and normalized histogram of the input image.

2-Determine the new gray levels using the histogram equalization transformation.

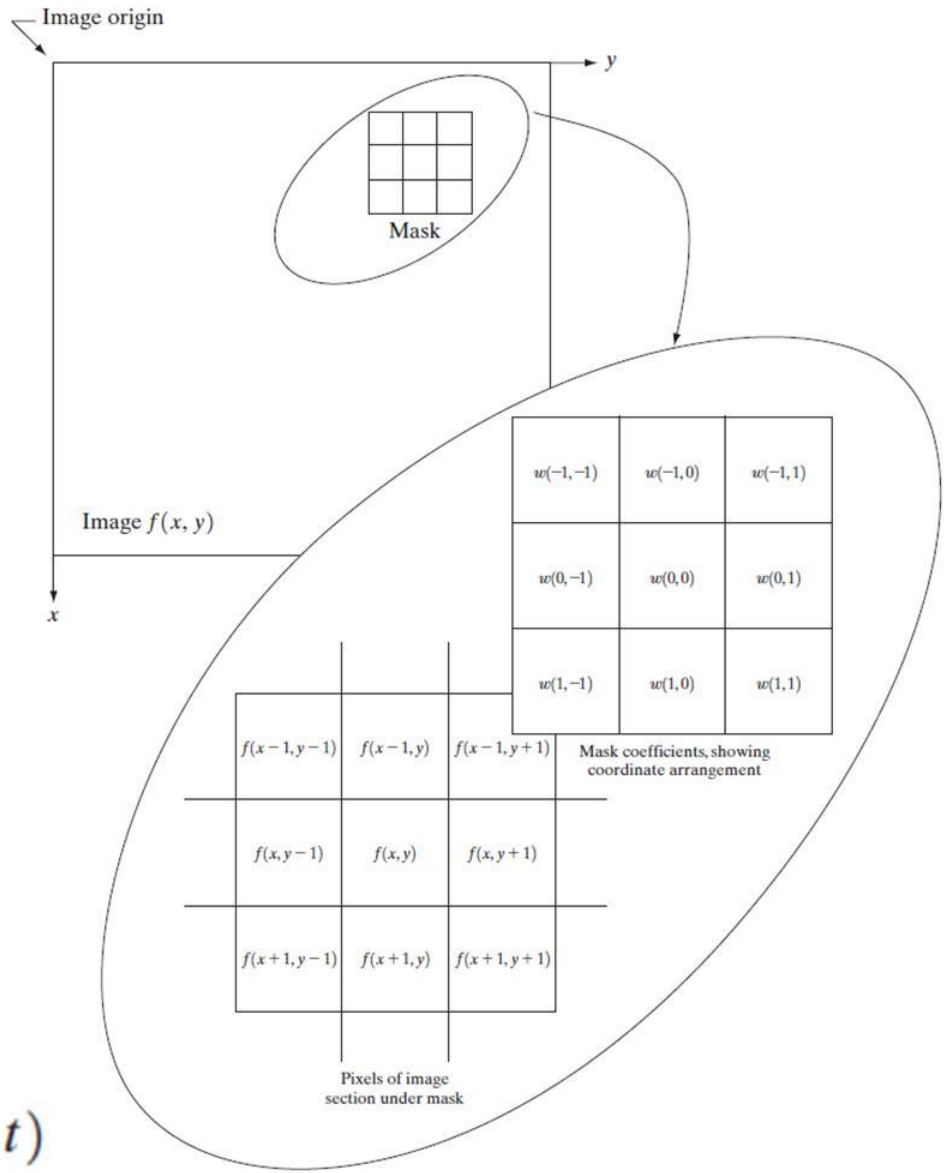
3-Write the resulting image matrix

4-Plot the histograms of both the original image and the histogram-equalized image.

Spatial Filtering

$w(-1,-1)$	$w(-1,0)$	$w(-1,1)$
$w(0,-1)$	$w(0,0)$	$w(0,1)$
$w(1,-1)$	$w(1,0)$	$w(1,1)$

Mask coefficients, showing coordinate arrangement



A spatial filter consists of (a) **a neighborhood**, and (b) **a predefined operation**

Linear spatial filtering of an image of size MxN with a filter of size mxn is given by the expression

$$g(x, y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t) f(x + s, y + t)$$

Spatial Filtering

Linear Spatial Filtering is also called
«Convolving a mask with an image»

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Convolution mask
(Convolution Kernel)

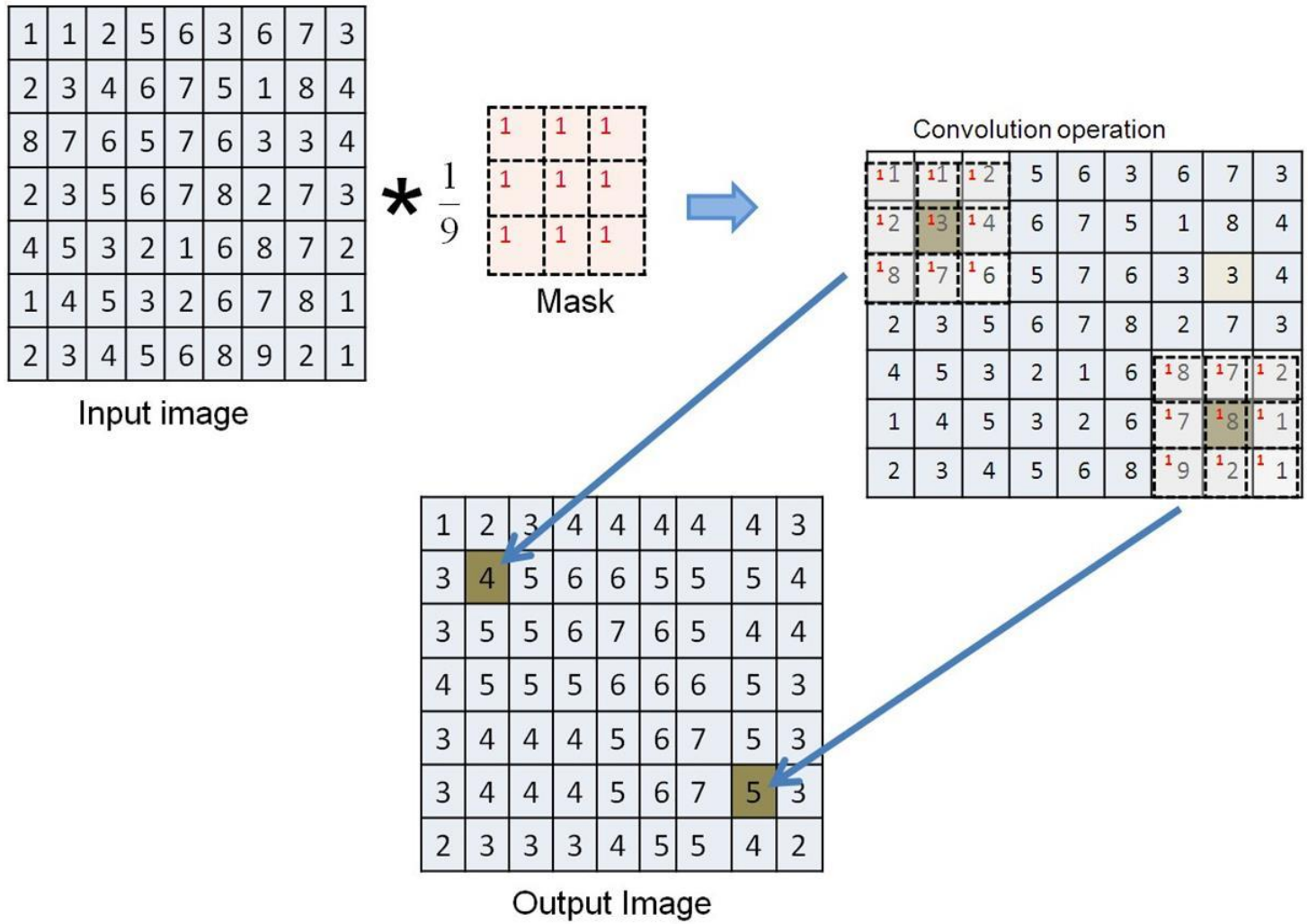
2D FIR filtering

- Mask filtering: convolution of the image with a 2D mask
- Applications to image enhancement:
  - Smoothing: low pass
  - Sharpening: high pass

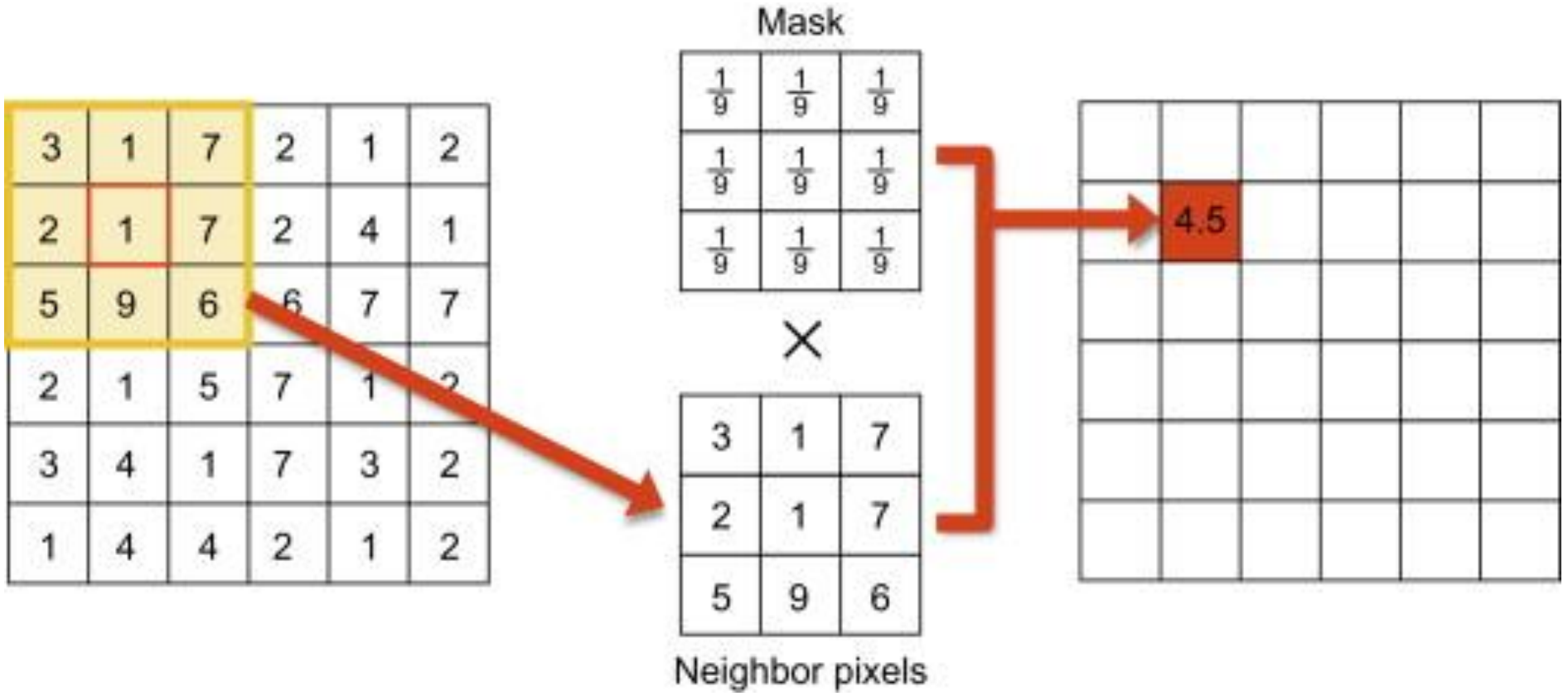
Data-dependent nonlinear filters

- Local histogram
- Order statistic filters
  - Median filter

Spatial Filtering



Spatial Filters



Handling border pixels

Zero padding : Input array values outside the bounds of the array are implicitly assumed to have the value 0.

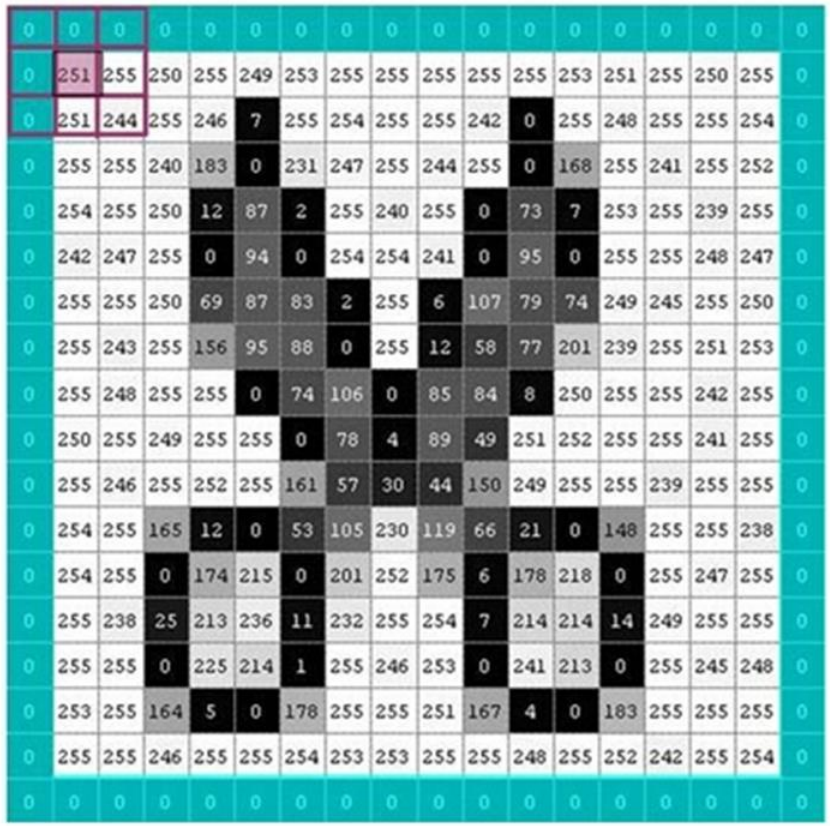
'symmetric‘: Input array values outside the bounds of the array are computed by mirror-reflecting the array across the array border.

'replicate' : Input array values outside the bounds of the array are assumed to equal the nearest array border value.

'circular' : Input array values outside the bounds of the array are computed by implicitly assuming the input array is periodic.

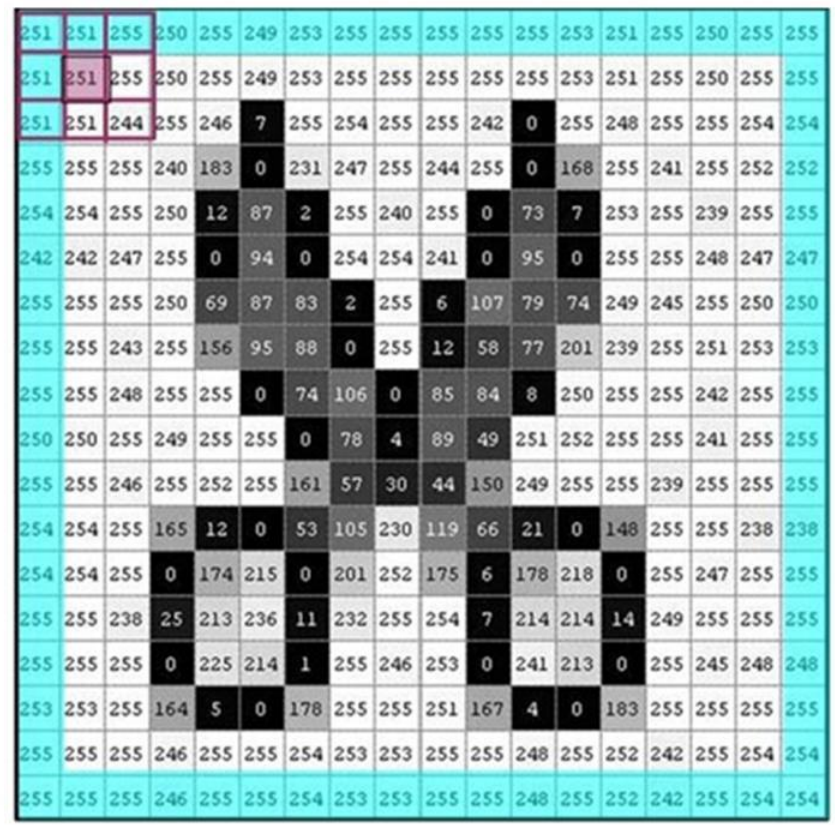
Handling border pixels

ZERO PADDING



0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	251	255	250	255	249	253	255	255	255	255	255	253	251	255	250	255	0	
0	251	244	255	246	7	255	254	255	255	242	0	255	248	255	255	254	0	
0	255	255	240	183	0	231	247	255	244	255	0	168	255	241	255	252	0	
0	254	255	250	12	87	2	255	240	255	0	73	7	253	255	239	255	0	
0	242	247	255	0	94	0	254	254	241	0	95	0	255	255	248	247	0	
0	255	255	250	69	87	83	2	255	6	107	79	74	249	245	255	250	0	
0	255	243	255	156	95	88	0	255	12	58	77	201	239	255	251	253	0	
0	255	248	255	255	0	74	106	0	85	84	8	250	255	255	242	255	0	
0	250	255	249	255	255	0	78	4	89	49	251	252	255	255	241	255	0	
0	255	246	255	252	255	161	57	30	44	150	249	255	255	239	255	255	0	
0	254	255	165	12	0	53	105	230	119	66	21	0	148	255	255	238	0	
0	254	255	0	174	215	0	201	252	175	6	178	218	0	255	247	255	0	
0	255	238	25	213	236	11	232	255	254	7	214	214	14	249	255	255	0	
0	255	255	0	225	214	1	255	246	253	0	241	213	0	255	245	248	0	
0	253	255	164	5	0	178	255	255	251	167	4	0	183	255	255	255	0	
0	255	255	246	255	255	254	253	253	255	255	248	255	252	242	255	254	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

REPLICATION



251	251	255	250	255	249	253	255	255	255	255	255	253	251	255	250	255	255	
251	251	255	250	255	249	253	255	255	255	255	255	253	251	255	250	255	255	
251	251	244	255	246	7	255	254	255	255	242	0	255	248	255	255	254	254	
255	255	255	240	183	0	231	247	255	244	255	0	168	255	241	255	252	252	
254	254	255	250	12	87	2	255	240	255	0	73	7	253	255	239	255	255	
242	242	247	255	0	94	0	254	254	241	0	95	0	255	255	248	247	247	
255	255	255	250	69	87	83	2	255	6	107	79	74	249	245	255	250	250	
255	255	243	255	156	95	88	0	255	12	58	77	201	239	255	251	253	253	
255	255	248	255	255	0	74	106	0	85	84	8	250	255	255	242	255	255	
250	250	255	249	255	255	0	78	4	89	49	251	252	255	255	241	255	255	
255	255	246	255	252	255	161	57	30	44	150	249	255	255	239	255	255	255	
254	254	255	165	12	0	53	105	230	119	66	21	0	148	255	255	238	238	
254	254	255	0	174	215	0	201	252	175	6	178	218	0	255	247	255	255	
255	255	238	25	213	236	11	232	255	254	7	214	214	14	249	255	255	255	
255	255	255	0	225	214	1	255	246	253	0	241	213	0	255	245	248	248	
253	253	255	164	5	0	178	255	255	251	167	4	0	183	255	255	255	255	
255	255	255	246	255	255	254	253	253	255	255	248	255	252	242	255	254	254	
255	255	255	246	255	255	254	253	253	255	255	248	255	252	242	255	254	254	

Smoothing Spatial Filters (Low pass filters)

- Smoothing filters are used for blurring and for noise reduction
- Blurring is used in removal of small details and bridging of small gaps in lines or curves
- Smoothing spatial filters include linear filters and nonlinear filters.

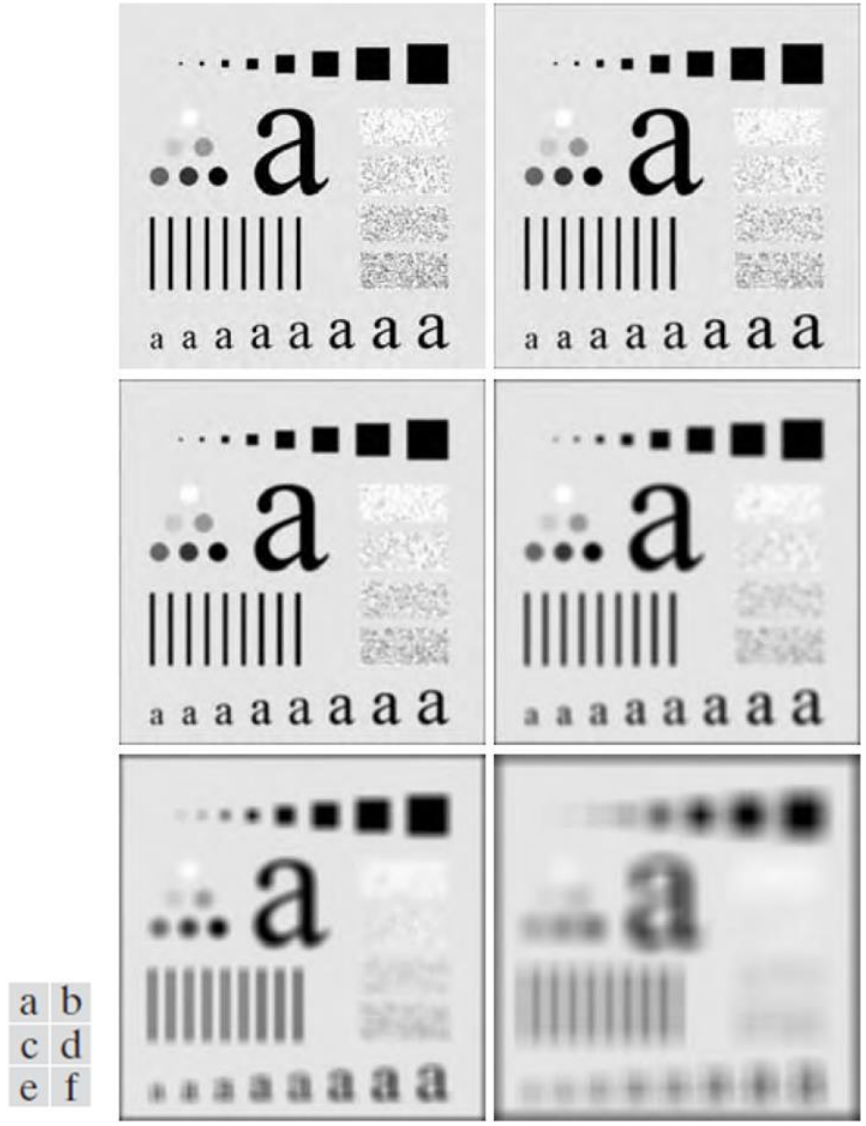
$$\frac{1}{9}\times\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}\qquad \frac{1}{16}\times\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix}$$

FIGURE 3.34 Two 3×3 smoothing (averaging) filter masks. The constant multiplier in front of each mask is equal to the sum of the values of its coefficients, as is required to compute an average.

Smoothing Spatial Filters

Averaging filter (Box filter)

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



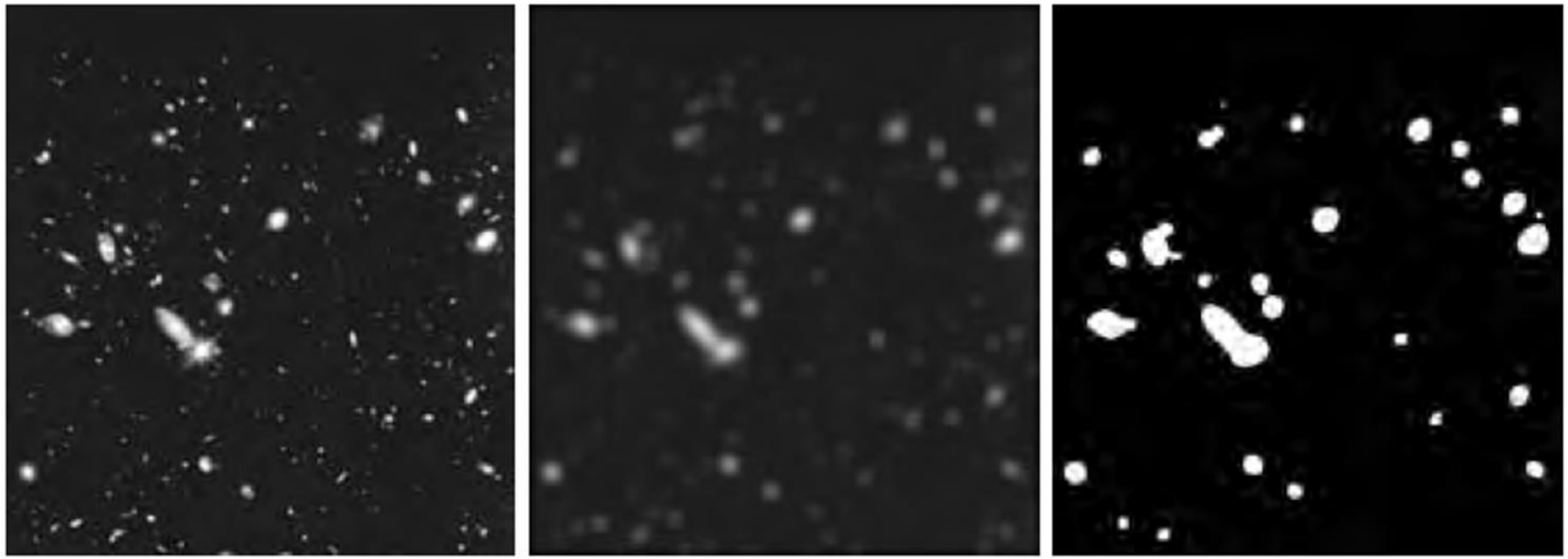
a b
c d
e f

FIGURE 3.35 (a) Original image, of size 500 × 500 pixels. (b)–(f) Results of smoothing with square averaging filter masks of sizes $n = 3, 5, 9, 15,$ and 35, respectively.

Smoothing Spatial Filters

Averaging filter (Box filter)

Excessive blurring is generally used to eliminate small objects from an image.
A number of objects have either blended with the background or their intensity has diminished considerably.
It is typical to apply thresholding to eliminate objects based on their intensity.



a b c

FIGURE 3.36 (a) Image from the Hubble Space Telescope. (b) Image processed by a 15 × 15 averaging mask. (c) Result of thresholding (b). (Original image courtesy of NASA.)

Smoothing Spatial Filters

Weighted average

$\frac{1}{16} \times$

1	2	1
2	4	2
1	2	1

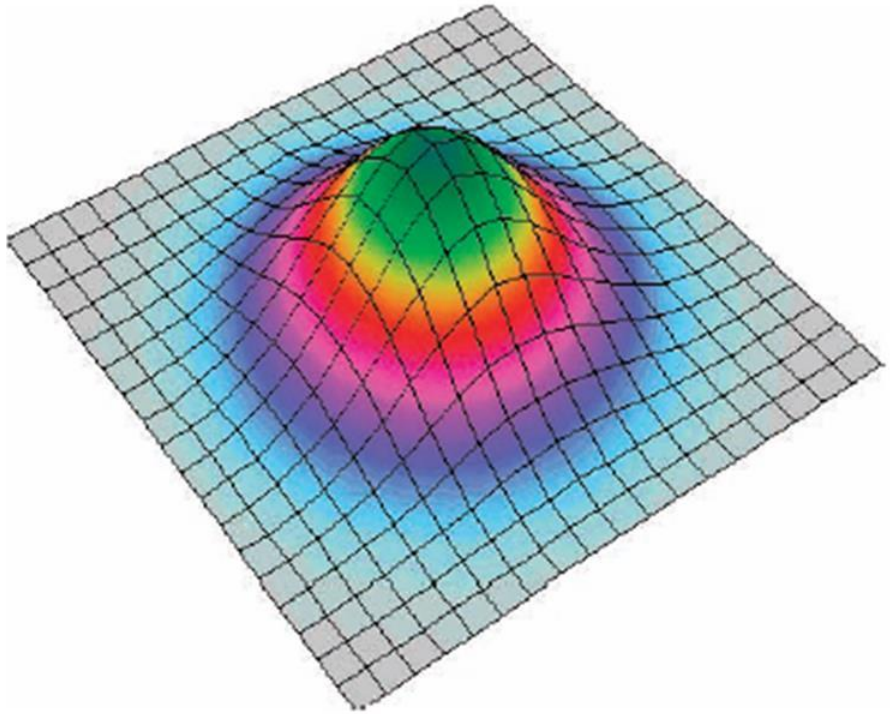
$$g(x, y) = \frac{\sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t) f(x + s, y + t)}{\sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t)}$$

Normalization of coefficient to ensure $0 \leq g(m,n) \leq L-1$

The pixel at the center of the mask is multiplied by a higher value than any other, thus giving this pixel more importance in the calculation of the average.
The other pixels are inversely weighted as a function of their distance from the center of the mask.
Helps to reduce blurring in the smoothing process.

Smoothing Spatial Filters

Gaussian Filter



σ = 1.0 pixels

0	0	1	1	1	0	0
0	2	8	14	8	2	0
1	8	37	61	37	8	1
1	14	61	100	61	14	1
1	8	37	61	37	8	1
0	2	8	14	8	2	0
0	0	1	1	1	0	0

σ = 1.5 pixels

0	0	1	2	3	2	1	0	0
0	2	6	11	14	11	6	2	0
1	6	17	33	41	33	17	6	1
2	11	33	64	80	64	33	11	2
3	14	41	80	100	80	41	14	3
2	11	33	64	80	64	33	11	2
1	6	17	33	41	33	17	6	1
0	2	6	11	14	11	6	2	0
0	0	1	2	3	2	1	0	0

MATLAB SESSION

```
Img = imread('penguins.jpg');
Img = rgb2gray(Img);


% Logarithmic transformation
Ad = double(Img);
B = log(Ad + 1);
B = uint8(255*B/max(B(:)));

% Gamma trasformation
gamma = 2;
Ad = double(Img);
Ad = Ad/max(Ad(:));
B = Ad.^gamma;
B = uint8(255*B);


% Thresholding
thr = 128;
B = 255*(Img > thr);
```



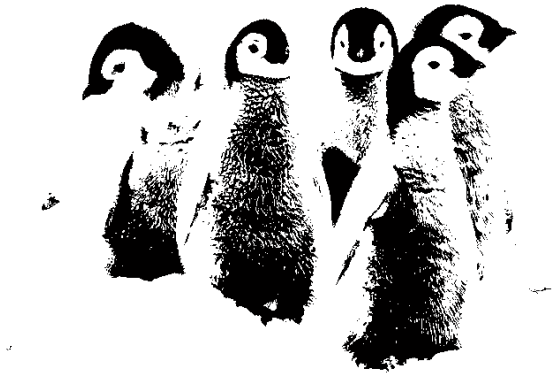
Original image



Log transformation



Gamma transformation
gamma = 2



Thresholding
threshold = 128

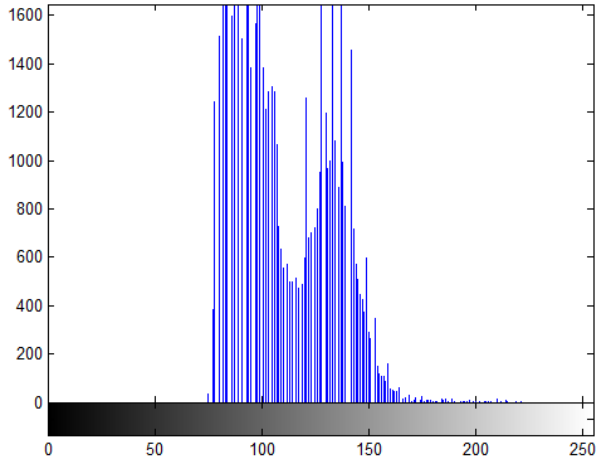
```
% Contrast Stretching with linear
% transformation

I = imread('pout.tif');
figure(1); imshow(I);
figure(2); imhist(I);

Id = double(I);
minI = min(Id(:));
maxI = max(Id(:));
Ic = 255/(maxI-minI)*(Id-minI);
Ic = uint8(Ic);
figure(3); imshow(Ic);
figure(4); imhist(Ic);
```



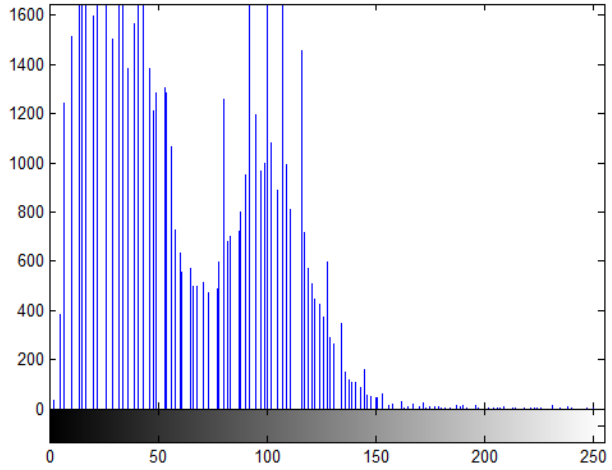
Original image



Histogram



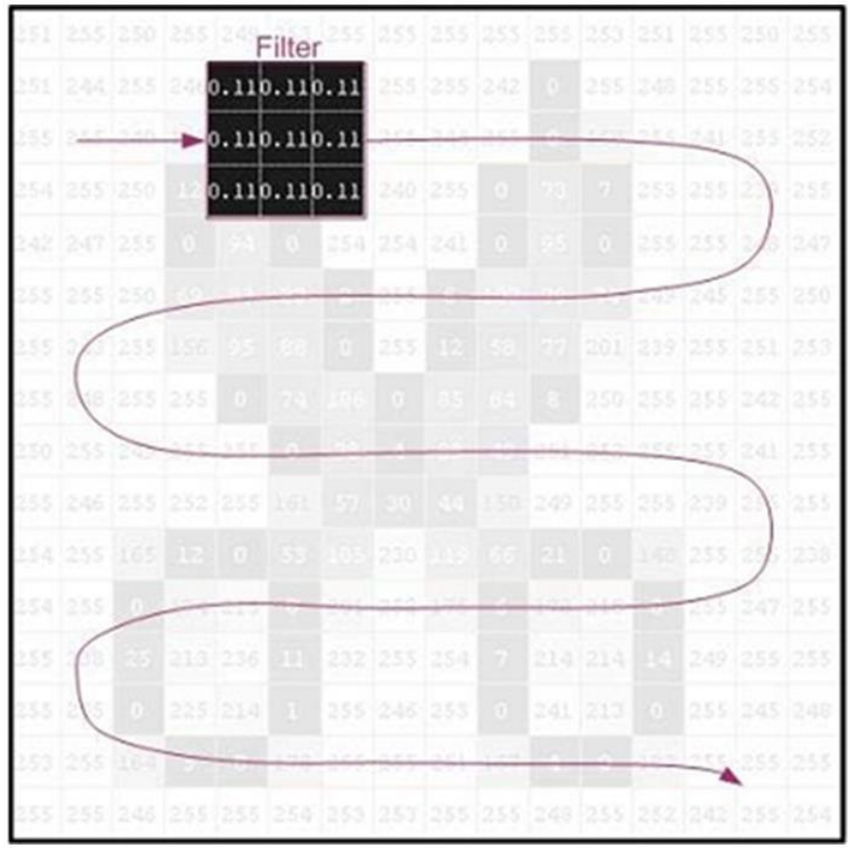
After contrast stretching



Resulting histogram

Transformation	Intensity Transformation Function
photographic negative	`neg=imcomplement(I);`
logarithmic	`I2=im2double(I);` `log=5*log(1+I2);`
gamma	`gamma=imadjust(I,[],[],0.4);`
Histogram equalization	`I_new = histeq(I);`

Image Filtering with MATLAB



- Size of neighbourhood = size of mask.
- Mask slides from left to right, top to bottom.
- Same operation is performed on every pixel.
- Neighbourhood exceeds image boundary = zero padding or replication of border pixel.

```
% 9x9 average filtering
% ignoring border pixels

A = imread('penguins.jpg');
A = rgb2gray(A);
A = double(A);
B = zeros(size(A));

mask = (1/81)*ones(9);

for i = 5:size(B,1)-4
    for j = 5:size(B,2)-4
        sub_image = A(i-4:i+4,j-4:j+4);
        s = sub_image.*mask;
        B(i,j) = sum(s(:));
    end;
end;

figure(1);
imshow(uint8(A));

figure(2);
imshow(uint8(B));
```



Original image



After average filtering with a 9x9 filter

```
% Sharpening with Laplace operator
% Zero-padding is applied for border pixels
A = imread('penguins.jpg');
A = rgb2gray(A);
figure; imshow(A);
A = double(A);

f_mask = [0 -1 0;-1 4 -1;0 -1 0];

Ab = zeros(size(A,1)+2,size(A,2)+2);
Ab(2:end-1,2:end-1) = A;
B = zeros(size(A));

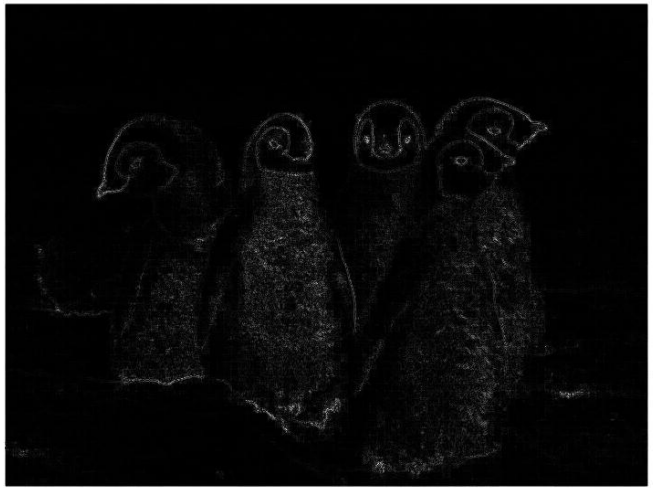
for i = 1:size(A,1);
    for j = 1:size(A,2);
        ij_window = Ab(i:i+2,j:j+2);
        a = ij_window.*f_mask;
        B(i,j) = sum(a(:));
    end;
end;

C = A + B;

figure; imshow(abs(B),[0 max(B(:))]);
figure; imshow(uint8(C));
```



Original image



Output of Laplace operator



Sharpened image

```
% Can use MATLAB's imfilter function for any spatial filter (kernel)

A = imread('penguins.jpg');
A = rgb2gray(A);
figure; imshow(A);
A = double(A);

f_mask = [0 -1 0;-1 4 -1;0 -1 0];

B = imfilter(A,f_mask,'replicate');

C = A + B;

figure
imshow(abs(B),[0 max(B(:))]);
figure
imshow(uint8(C));
```